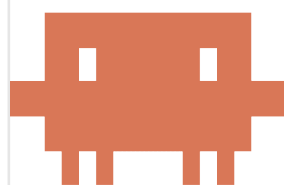
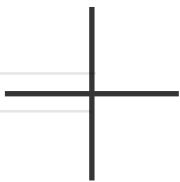




urban.pts.v



//integrated multimodal
mobility infrastructure
model for
post-car cities



prompt city::urban vision wolfsburg::sose26

m.sc. iudd
bauhaus.institute for
experimental architecture
bauhaus-universität weimar

anastasiia mulyndina
basak pinar
ömer faruk aslan

Our Streets are for people

Wolfsburg was built for the car. Its streets, its squares and its ground are all measured out for driving and for storage. The city gave its best space to machines that stand still most of the day.

Land that could hold homes, gardens, a place to gather sits captured beneath parking. The public realm became a corridor to drive through, **not a place to be.**

A city shaped this way asks every resident to own a car simply to take part in it. Convenience for some became dependence for all.

We intend to take that ground back. To reclaim the space the car has held, and return it to the people of the city.

Parking becomes public space. Traffic lanes become room to walk, to plant, to live. The land the car captured is given back for housing, for greenery, for shared urban life.

And in place of the private car, a way to move together: collective, open to all, reducing what each of us must own in order to belong.

Less private traffic. More common ground.

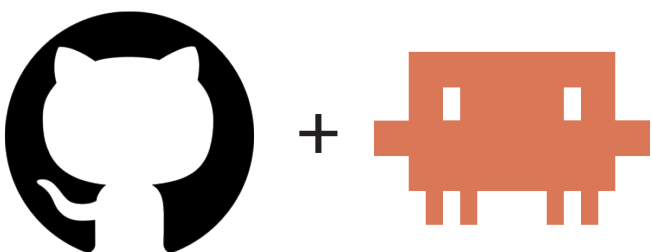
Urban Isolation

//definition
segregated. isolated. privatized.

Wolfsburg is a city built around the automobile infrastructure. Volkswagen's production has shaped not only the economy but also the city's spatial logic.

Wide roads, fragmented districts, villages disconnected from one another. The result is social disintegration and total dependence on private cars.

analyzer*is*



//interactive
spatial
analysis
tool

check it out:
[https://github.com/
annestasiia/wolfsburg-
activity-map](https://github.com/annestasiia/wolfsburg-activity-map)

Urbanscape

//definition

integrated. accessible. social.

Wolfsburg gives a disproportionate amount of urban land to private cars: parking lots, surface parking, on street parking, and car-oriented street infrastructure. This is amplified by Volkswagen's commuting patterns: the city was literally built around car storage and car movement.

The project argues this land-use model is no longer necessary. If mobility is reorganized around a **shared, multimodal, autonomous system**, the land currently used for parking becomes available for public life, greenery, housing, and public space.

//possible outcomes accessible.		
Shared vehicles (in fleet)	1300	641 e-bikes 55 shuttle pods 33 buses 369 micro-pods 175 EVs
Trips served (per day)	104.000	17.400 residents (city center) 18.000 workers (daily commuters) 7.080 daily visitors
Money saved (per household/yr)	7.596€	727€/mo private car -> 94€/mo shared mobility
[ADAC 2024, MOIA Hamburg]		87% cost reduction

//hub system integrated.

A three-tier shared mobility network enabling seamless mode-chaining across the city with e-bikes, autonomous shuttles, and shared pods. **Without a private vehicle.**

Hub network

By introducing the hub network, the city gains the opportunity to become progressively more connected, beginning with districts closest to the centre, and gradually extending mobility coverage to more remote settlements as the network expands. The hub system makes different modes of mobility structurally available across all districts, reducing car dependency at the network level.



External Flows

Integrating external commuter flows into the hub network is critical for reducing the volume of private vehicles entering the city. By positioning Hub L nodes at the main entry points, the system creates a seamless transition from regional transport to the internal shared mobility network — making it practical for Einpendler to leave their cars outside the city boundary and complete their journey by shared mobility means.

//hub system
social.

Facility network

The facility network analysis shows that proposed hub locations substantially improve non-motorised access to urban amenities across all districts. Areas where current walking and cycling accessibility scores are lowest — peripheral residential districts — benefit most from hub-mediated connectivity. Hub placement transforms mobility gaps into connected catchments, enabling residents to reach everyday destinations without a private vehicle.



maestro

an

top

te

st

ma

st

ma

Post-Car Future of Wolfsburg – Project Workspace

A shared workspace for our urban-design project **"Post-Car Future of Wolfsburg"** – reimagining Wolfsburg as a **post-private-car city** where the organizing element of mobility shifts from **parking** → **mobility hubs**, freeing car-dominated land for public life.

> Studio project, Bauhaus-Universität Weimar ("Prompt City" studio)

What this is

This repository is the **single source of truth** for the project: the concept, the locked decisions, the research that backs our arguments, the consultation history, the 3D/Rhino workflow, and a memory system so work continues smoothly across sessions and between the three of us.

It is **not** the final deliverable – it's the workshop behind it. The final outputs (maps, drawings, the competition submission) are produced **from** this material.

What's inside

Folder / file | what it holds |

---|---|

CLAUDE.md | The project briefing – read this first. Thesis, locked decisions, folder map, working protocol. |

HANDOFF.md | "Where are we right now" – current state, deadlines, and the next actions. **Read this second.** |

project/ | The living project: **current.md** (latest description), **deliverables.md**, **hub_typologies.md**, **rhino_toolpalette.md**. |

research/ | **Research Library** – 10 cited documents (arguments, weak-points, hub typologies, mobility, precedents, behavioral shift, theory, future, workflow) + a 199-source bibliography. |

decisions.md | Formally locked design decisions, with reasoning. |

notes.md | Running scratchpad of ideas and quick decisions. |

tasks.md | Work tracker (open / in-progress / done). |

briefs/ | What we must respond to: competition brief, studio brief, midterm brief. |

consultations/ | One file per tutor consultation – feedback and decisions. |

sources/ | Precedents and data sources. |

sessions/ | Log of every working session (**INDEX.md** + one folder per session) – our memory across time. |

rhino/ | Bridge to drive our Rhino 8 model from the terminal (the 45-block **"kit-of-parts"**). |

web-tool/ | Status of the **Wolfsburg Activity Map** (the interactive analysis tool). |

> **The web tool itself** lives in its own repository: **[annestasia/wolfsburg-activity-map]**(<https://github.com/annestasia/wolfsburg-activity-map>) – React + Maplibre, 7 analysis modes (mobility, facilities, greenery, intermodal-hub placement, cycling network, satellite, data). It's excluded here to avoid duplicating it.

What it offers

- **A defensible argument bank** – every claim backed by a cited number or real precedent (e.g. Wolfsburg has Germany's highest car density, 956 cars / 1,000 residents).
- **A red-team risk register** – the hardest jury objections (AV realism, fleet math, Vw-company-town politics, financing) each with a counter and a concrete fix.
- **A precedent library** – Groningen, Ghent, Houten, Barcelona, Paris, Bremen mobility hubs.. with hard outcome numbers and what's transferable.
- **A realistic production plan** to June 25 / July 9-12 / Aug 16, plus Rhino and AI/agentic workflow guidance.
- **A memory system** so a new session (or teammate) can resume in minutes instead of starting cold.

How to use this system

Starting a work session (the 60-second resume):

1. Open **CLAUDE.md** → the project briefing.
2. Open **HANDOFF.md** → what's done, what's next, what's blocked.
3. Need an argument, number, or precedent? → **research/README.md** is the index.
4. Do the work.

Ending a work session (so nothing is lost):

1. Write a session log – copy **sessions/_template.md** to **sessions/YYYY-MM-DD_topic/notes.md**.
2. Add a line to **sessions/INDEX.md**.
3. Update **HANDOFF.md** (state + next actions) and, if a decision was locked, **decisions.md**.

For the 3D model (Rhino): see **rhino/README.md** – Rhino 8 must be open with **'mcpstart'**, then the model is editable from the terminal.

The project in one minute

- **Thesis:** Parking City → Hub City. Replace privately-owned, idle, space-hungry cars with a shared, multimodal, partly-autonomous system organized around hubs.
- **The network:** 68 hubs in three tiers – 6 large (repurposed multi-story car parks), 19 Medium, 43 Small.
- **The fleet:** 763 shared vehicles (e-bikes, shuttle pods, autonomous buses, micro-pods, shared EVs), sized for ~100,000 trips/day.
- **The proof case:** the Volkswagen factory shift wave – ~10,000 workers arriving in ~1 hour is **predictable**, concentrated demand, making Wolfsburg an unusually strong prototype for shared mobility.

Wolfsburg Activity Map

An interactive browser tool for the **Post-Car Future of Wolfsburg** urban-design project. It pulls geographic data from OpenStreetMap, runs spatial-analysis algorithms in the browser, and visualises the results on a map – to help design and **prove** a city organised around mobility hubs instead of parking.

Built with **React 18 + Vite**, **Maplibre GL JS**, **Zustand**, and **Tailwind CSS**. No backend.

> Detailed, per-analysis documentation (what each tool measures and how the algorithm works) lives in
> **docs/MODES.md**. Architecture and contributor guidance for future work
> (including AI-assisted sessions) live in **CLAUDE.md**.

Quick start

```
``bash
npm install      # first time only
npm run dev      # starts Vite dev server
````
```

Open the address it prints (usually <http://localhost:5173>).

```
``bash
npm run build # production build → dist/
npm run preview # preview the production build locally
````
```

Deployment

Pushing to the **master** branch triggers `.github/workflows/deploy.yml`, which runs `npm ci && npm run build` and publishes `dist/` to **GitHub Pages**. No manual deploy step.

What’s in it – the five sections

The app opens on a landing pitch; pick a section from the right-hand navigation. The numbered sections are:

- # | Section | What it does | Status |
- |-----|-----|-----|
- 01 | **Post-Car Strategy** | Slide-style narrative of the overall vision.
- 02 | **Capacity Analysis** | Fleet-sizing / modal-distribution dashboard (the quantitative proof).
- 03 | **Hub System** | The map-analysis toolkit, with two sub-areas (below).

Data

Data | Source | Loaded |
-----|-----|-----|
District boundaries, venues, parks, water, forest, buildings | Pre-processed JSON in `src/data/` | Bundled, loaded on startup |
Roads, footways, bus stops, car/bike parking, facilities, historic sites, cycling, parks/forests | `public/*.geojson` (pre-fetched from OSM) | `fetch()` on startup |
Venue activity & hours | Team **Excel** | `src/data/venues.xlsx`, geocoded via Nominatim | Manual upload (Facilities) |
Population & mobility stats | Mid 2017 + VW workforce data | Static, in `analysis/` |
Base map tiles | **CARTO** positron | `basemaps.cartocdn.com`, no API key | Live |

To refresh the OSM extracts or regenerate the fleet/hub numbers, see the **Data-refresh guide** in `[CLAUDE.md](CLAUDE.md)`.

Excel file format (Facilities mode)

The app expects an `src/data/facilities.xlsx` with data starting on **row 4** (rows 1–3 are headers/metadata):

| Col | Field | Col | Field |
|-----|---------------------------|-----|--|
| 1 | District | 4 | Activity intensity |
| 2 | Category | 5 | Age groups |
| 3 | Name | 6 | Google rating |
| 4 | Type | 7 | Notes |
| 5 | Street address | 8 | Mon-Sun activity (<code>Low</code> / <code>Med</code> / <code>High</code>) |
| 6 | City | 9 | |
| 7 | Opening hours (free text) | 10 | |
| 8 | Peak times | 11 | |

Addresses are geocoded via Nominatim (1 req/sec) and cached in `localStorage`.

Resetting cached data

```
``js
// Browser console:
localStorage.removeItem('wolfsburg_geocache_v1') // forces re-geocode on next upload
// District boundary caches are keyed 'wolfsburg_boundary_v1<DistrictName>'
// clear the whole app's localStorage to reset everything:
Object.keys(localStorage).filter(k => k.startsWith('wolfsburg_')).forEach(k => localStorage.removeItem(k))
````
```

---

## Project context

```
GHPython – Balanced hub network: centre local mesh + peripheral feeders.
Centre S/M hubs form a dense LOCAL mesh (k-nearest); outer S hubs feed INWARD
to the nearest interchange. Both layers separate & tunable so neither dominates.
Self-contained, no file. Paste into a Python 3 Script component.
#
INPUTS (all optional):
centre_k int (Item) each centre hub links to its k nearest centre peers.
Default 3.
to_L_only bool (Item) outer feeder target: True = nearest L only, False =
nearest L/M. Default False.
max_reach float(Item) cap (m) on feeder length; longer ones dropped. De-
fault 0 = no cap.
#
OUTPUTS:
centre_net Curve centre S+M local mesh (short links)
outer_feeders Curve each outer S -> nearest interchange
feeder_dist float feeder lengths (m), same order -> color by this
(long = weakly served)
centre_points Point3d centre S+M hubs
outer_s_points Point3d peripheral S hubs
interchange_pts Point3d L/M targets used by feeders
#
Coords LOCAL to Rhino origin (E=621915.018, N=5808262.547).
import Rhino.Geometry as rg

(tier, zone, rhino_x, rhino_y)
DATA = [
 („L“,“outer“, -2815.936,278.799),
 („L“,“centre“, -3271.107,2654.392),
 („L“,“centre“, 231.835,2874.57),
 („L“,“outer“, -3469.871,-1198.89),
 („L“,“outer“, -2448.865,-2626.189),
 („M“,“centre“, -600.495,875.696),
 („M“,“centre“, -355.36,1930.559),
 („M“,“centre“, 1112.494,1750.464),
 („M“,“centre“, 302.23,2069.376),
 („S“,“centre“, -448.417,1925.787),
 („S“,“centre“, 318.472,1642.621),
 („S“,“centre“, 495.578,1279.352),
 („S“,“centre“, -14.723,1884.224),
 („S“,“centre“, -14.99,1274.629),
 („S“,“centre“, -223.789,1553.098),
 („S“,“centre“, -885.536,1305.517),
 („S“,“centre“, -453.721,603.246),
 („S“,“centre“, -1143.23,1766.672),
 („S“,“centre“, -385.214,1129.147),
 („S“,“centre“, 36.695,632.482),
 („S“,“centre“, -1276.47,1400.111),
 („S“,“outer“, 632.953,865.264),
 („S“,“centre“, -983.918,978.878),
 („S“,“centre“, 859.856,1188.181),
 („S“,“outer“, 282.85,2356.545),
 („S“,“centre“, 783.893,1801.229),
 („S“,“outer“, 3041.612,-1479.198),
]
```

```
def _i(name, dflt):
 v = globals().get(name, None)
 try: return int(v)
 except (TypeError, ValueError): return dflt
def _b(name):
 v = globals().get(name, None)
 return bool(v) if v is not None else False
def _f(name, dflt):
 v = globals().get(name, None)
 try: return float(v)
 except (TypeError, ValueError): return dflt

k = max(1, _i(„centre_k“, 3))
to_L_only = _b(„to_L_only“)
cap = _f(„max_reach“, 0.0)

P = lambda x, y: rg.Point3d(float(x), float(y), 0.0)

centre_points = [P(x, y) for (t, z, x, y) in DATA if z == „centre“ and t
in („S“, „M“)]
outer_s_points = [P(x, y) for (t, z, x, y) in DATA if t == „S“ and z ==
„outer“]
if to_L_only:
 interchange_pts = [P(x, y) for (t, z, x, y) in DATA if t == „L“]
else:
 interchange_pts = [P(x, y) for (t, z, x, y) in DATA if t in („L“,
„M“)]

---- centre local mesh: k-nearest among centre peers (deduplicated) ----
centre_net = []
seen = set()
n = len(centre_points)
for i in range(n):
 pi = centre_points[i]
 order = sorted(range(n), key=lambda j: pi.DistanceTo(centre_
points[j]))
 cnt = 0
 for j in order:
 if j == i:
 continue
 key = (i, j) if i < j else (j, i)
 if key not in seen:
 seen.add(key)
 centre_net.append(rg.LineCurve(pi, centre_points[j]))
 cnt += 1
 if cnt >= k:
 break

---- peripheral feeders: each outer S -> nearest interchange ----
outer_feeders, feeder_dist = [], []
for s in outer_s_points:
 best, bd = None, 1e18
 for q in interchange_pts:
 dd = s.DistanceTo(q)
 if dd < bd:
 bd, best = dd, q
]
```



```

GHPython component – Load Hub M & S points into Rhino
Bulletproof against encoding (utf-8/utf-16/cp1252), line endings (LF/CRLF/CR),
delimiter (comma/semicolon/tab) and German decimal commas – i.e. survives being
re-saved by Excel / Numbers.
#
INPUT (one input param):
csv_path str (Item) -> path to the hubs CSV
OUTPUTS (four output params):
m_points Point3d
m_names str
s_points Point3d
s_names str
#
Coords are LOCAL to the Rhino origin (E=621915.018, N=5808262.547).

import csv, io
import Rhino.Geometry as rg

m_points, m_names, s_points, s_names = [], [], [], []

def _read_text(path):
 with io.open(path, „rb“) as f:
 raw = f.read()
 if raw[:2] == b“PK“: # ZIP-based doc: .numbers / .xlsx / .pages – NOT a CSV
 raise ValueError(
 „This file is a Numbers/Excel document (zip package), not a CSV. “
 „In Numbers use File > Export To > CSV, or download a fresh copy of “
 „wolfsburg_hubs_MS_centre_utm.csv and connect it WITHOUT opening it.“)
 if raw[:2] in (b“\xff\xfe“, b“\xfe\xff“):
 t = raw.decode(„utf-16“, „replace“)
 elif raw[:3] == b“\xef\xbb\xbf“:
 t = raw[3:].decode(„utf-8“, „replace“)
 elif b“\x00“ in raw[:400]:
 t = raw.decode(„utf-16-be“ if raw[:1] == b“\x00“ else „utf-16-le“, „repla-
ce“)
 else:
 t = None
 for enc in („utf-8“, „cp1252“, „latin-1“):
 try:
 t = raw.decode(enc); break
 except UnicodeDecodeError:
 pass
 if t is None:
 t = raw.decode(„utf-8“, „replace“)
 return t.replace(„\x00“, „“).replace(„\uffff“, „“)

def _num(s):
 s = (s or „“).strip()
 try:
 return float(s)
 except ValueError:
 return float(s.replace(„“, „.“)) # German decimal comma

```

```

total = 0
delim = „“
if not csv_path:
 print(„csv_path is empty – check the input connection / panel.“)
else:
 lines = _read_text(csv_path).splitlines()
 header = next((ln for ln in lines if ln.strip()), „“)
 delim = max([„“, „;“, „\t“], key=lambda d: header.count(d)) # de-
tect delimiter
 reader = csv.DictReader(lines, delimiter=delim)
 cols = reader.fieldnames or []
 if „tier“ not in cols or „rhino_x“ not in cols:
 print(„WRONG FILE: columns are %s“ % cols)
 print(„Expected: tier,id,name,lng,lat,easting,northing,rhi-
no_x,rhino_y,meta“)
 print(„-> point csv_path at wolfsburg_hubs_MS_centre_utm.csv (30
rows)“)
 for row in reader:
 total += 1
 try:
 x = _num(row[„rhino_x“]); y = _num(row[„rhino_y“])
 except (KeyError, ValueError, TypeError):
 continue
 pt = rg.Point3d(x, y, 0)
 tier = (row.get(„tier“) or „“).strip()
 name = row.get(„name“) or „“
 if tier == „M“:
 m_points.append(pt); m_names.append(name)
 elif tier == „S“:
 s_points.append(pt); s_names.append(name)

print(„delimiter=%r rows=%d -> Loaded M=%d S=%d“
 % (delim, total, len(m_points), len(s_points)))

```